UNIVERSIDAD
Panamericana

PROGRAMACIÓN AVANZADA

Sesión 08

Implementación en OmegaUP

→ NOTAS DE CLASE

Universidad Panamericana
Dr. Miguel Alcaraz Rivera • malcaraz@up.edu.mx

2026

Índice

1 El juez en línea — recordatorio rápido **2**

2 La sesión de hoy **3**

3 Problema 1 — Separación de bits: posiciones pares e impares **4**
3.1 Enunciado resumido 4
3.2 Ejemplo 4
3.3 Clave 4

4 Problema 2 — Separación de bits: enumerar 1s **5**
4.1 Enunciado resumido 5
4.2 Ejemplo 5
4.3 Clave 5

5 Problema 3 — Chasquidos Mágicos **6**
5.1 Enunciado resumido 6
5.2 Ejemplo 6
5.3 Clave 6

6 Problema 4 — Los bits de la clase de programación **7**
6.1 Enunciado resumido 7
6.2 Ejemplo 7
6.3 Clave 7

7 Resumen **7**

El juez en línea — recordatorio rápido

Ya viste esto a detalle en la sesión 02. El resumen que necesitas hoy:

Un **juez en línea** compila tu código, lo ejecuta contra casos de prueba ocultos y te devuelve un veredicto automático. No hay revisor humano: o tu programa produce exactamente la salida correcta, o no.

Veredicto	Qué pasó	Qué hacer
`AC` — Accepted	Correcto en todos los casos	Listo, pasa al siguiente
`WA` — Wrong Answer	Falla en algún caso	Busca el caso que rompe tu lógica
`CE` — Compilation Error	No compila	Lee el error del compilador
`TLE` — Time Limit	Correcto pero lento	Busca una solución con menos pasos
`RE` — Runtime Error	El programa se cae	Revisa división por cero, acceso fuera de rango

El juez es **literal**: un espacio de más, un salto de línea faltante o una mayúscula incorrecta producen `WA`. Lee la sección de salida del enunciado con mucho cuidado antes de someter.

Flujo de trabajo recomendado:

1. Implementa la función en el cuaderno de Jupyter (sesión 08).
2. Verifica que todos los casos de la celda de prueba pasen.
3. Escribe el programa completo con `cin`/`cout` y pruébalo con el ejemplo del enunciado.
4. Somete a OmegaUP.

La sesión de hoy

Trabajo en **equipo de tres**. El objetivo es obtener `AC` en los cuatro problemas de la sesión anterior. Cada problema vale puntos independientes: si no terminas uno, pasa al siguiente.

✔ **Recuerda**

Patrón de entrada/salida en C++ para el juez:

- Lee con `cin`, imprime con `cout`.
- No imprimas mensajes extra como “Ingresa un número:”.
- El juez redirige `stdin`/`stdout`; no uses archivos.
- Para leer hasta 0: `while (cin >> n && n != 0)`.
- Para leer hasta fin de archivo: `while (cin >> n)`.

Problema 1 — Separación de bits: posiciones pares e impares

• Enunciado resumido

Dado n, calcula:

- a: los bits de n en posición **impar** (bits 1, 3, 5, . . .), todos los demás en 0.
- b: los bits de n en posición **par** (bits 0, 2, 4, . . .), todos los demás en 0.

Leer enteros hasta recibir un 0. Imprimir `a b` por cada número.

• Ejemplo

n	Binario	a	b	Salida
13	`1101`	8	5	`8 5`
6	`0110`	2	4	`2 4`
7	`0111`	2	5	`2 5`

• Clave

Usa una **máscara AND**. La constante `0xAAAAAAAA` tiene encendidos exactamente los bits en posición impar; `0x55555555` tiene los de posición par. La solución completa cabe en dos líneas.

Verifica tu solución con `n = 55`: el resultado correcto es `34 21`. Si obtienes otro valor, revisa qué máscara usas para a y cuál para b.

Problema 2 — Separación de bits: enumerar 1s

• Enunciado resumido

Dado n, enumera sus bits encendidos de **derecha a izquierda** empezando en 1. Los 1s con índice impar (1.°, 3.°, . . .) forman a; los de índice par (2.°, 4.°, . . .) forman b. Cada bit conserva su posición original.

Leer enteros hasta 0. Imprimir `a b`.

• Ejemplo

n	Binario	Enumeración de 1s	a	b
13	`1101`	bit0=1 (1°), bit2=1 (2°), bit3=1 (3°)	9	4
6	`0110`	bit1=1 (1°), bit2=1 (2°)	2	4
7	`0111`	bit0=1 (1°), bit1=1 (2°), bit2=1 (3°)	5	2

• Clave

Recorre los bits de n de derecha a izquierda con un desplazamiento. Lleva un contador que incrementa cada vez que encuentras un 1; si el contador es impar, ese bit va a a; si es par, a b.

Empieza con `int conteo = 0, a = 0, b = 0;` y un bucle que avanza bit a bit. Para cada bit encendido, incrementa `conteo` y decide con `conteo % 2`.

Problema 3 — Chasquidos Mágicos

• Enunciado resumido

n enchufes en serie, todos apagados. Cada chasquido cambia el estado de los enchufes que tienen corriente. ¿Está la lámpara (enchufe n) prendida después de c chasquidos?

Entrada: dos números n y c por línea ($1 \leq n \leq 30$, $0 \leq c \leq 10^8$). Salida: `PRENDIDA` o `APAGADA`.

• Ejemplo

n	c	Salida
1	0	`APAGADA`
1	1	`PRENDIDA`
4	0	`APAGADA`
4	47	`PRENDIDA`

• Clave

La cadena de enchufes es un **contador binario**: después de c chasquidos, el enchufe i tiene el mismo estado que el bit $i-1$ de c. La lámpara (enchufe n) está prendida si y solo si el bit $n-1$ de c vale 1:

```
(c » (n-1)) & 1
```

El programa completo cabe en una sola línea de `cout`.

> c puede llegar a 10^8. Declara c como `long long` o `unsigned int` para evitar desbordamientos con el desplazamiento.

Problema 4 — Los bits de la clase de programación

• Enunciado resumido

Dado un mapa de bits A (cadena de caracteres `'0'` y `'1'`), construir S extrayendo bits del centro:

- Long. impar: extrae el bit central, agrégalo al final de S.
- Long. par: extrae los dos bits del centro; agrega el **mayor primero**, luego el menor; ambos salen de A.

Entrada: primera línea con k (número de casos); luego k cadenas. Salida: `Caso #x: y` para cada caso. Si $y > 10^9 + 7$, imprime $y \bmod (10^9 + 7)$.

• Ejemplo

A	Salida
`00000`	`Caso #1: 0`
`0101101`	`Caso #2: 106`
`100`	`Caso #3: 2`

• Clave

Simula el proceso directamente. Guarda A en un `string` (o `deque<char>`). En cada paso calcula el índice del centro con `size() / 2` y usa `erase()` para extraer.

Acumula S **como número**, no como cadena:

Caso	**Actualización de S**
Long. impar (un bit extraído b)	$S = (S \cdot 2 + b) \bmod M$
Long. par (mayor b_1, menor b_2)	$S = (S \cdot 4 + b_1 \cdot 2 + b_2) \bmod M$

Al extraer el par, los índices del centro izquierdo y derecho en una cadena de longitud n par son $n/2 - 1$ y $n/2$ (indexando desde 0). Extrae primero el derecho y luego el izquierdo para que el índice no se desplace entre las dos operaciones de `erase`.

Resumen

Problema	**Solución en una línea**
Sep. pares/imp.	`a = n & 0xAAAAAAAA; b = n & 0x55555555`
Sep. enum. 1s	Bucle bit a bit; alternar entre a y b según paridad del conteo
Chasquidos	`(c >> (n-1)) & 1`
Alg. de Phil	Simulación con `string::erase`; acumular S con módulo